Memoria del proyecto · Actividad 1

Proyecto: Taskflow — gestión visual de tareas

Tecnología: Ionic y Angular con Supabase

Fecha de corte: 27 de octubre de 2026

Estado: Autenticación y lectura/actualización de tareas conectadas; CRUD restante pendiente

1. Resumen ejecutivo

Se construyó una primera versión visual de Taskflow, un panel adaptable para consultar tareas, buscar y filtrar por estado, y visualizar métricas mediante gráficos SVG de líneas y de dona. Se ampliaron las pruebas automatizadas y se corrigieron aspectos de accesibilidad y rendimiento.

También se integró el cliente oficial de Supabase con configuración local, registro e inicio de sesión, protección de la ruta privada, lectura de tareas e historial, actualización de estado y sincronización Realtime. El usuario confirmó que ejecutó la migración SQL que restringe las inserciones directas de auditoría. La URL y la clave pública locales se validaron sin incluirlas en este documento; los endpoints de salud de autenticación y consulta de tareas respondieron HTTP 200.

Importante: se ha conectado la lectura y actualización del estado de las tareas, pero todavía no están implementados todos los flujos CRUD (en particular crear y eliminar). El gráfico semanal continúa siendo ilustrativo. Aunque el usuario confirma que aplicó la migración de auditoría, su ejecución remota no se pudo inspeccionar directamente desde este entorno.

2. Contexto y requisitos considerados

- Aplicación visual de gestión de tareas.
- Uso de Supabase como servicio de backend.
- Tablas con columnas tipadas y sin claves foráneas, según el requisito indicado para el trabajo.
- Seguridad mediante Row Level Security (RLS).
- Interfaz en español, adaptable y accesible.
- Gráficos distintos de barras.

PostgreSQL sigue siendo un motor relacional aunque se creen tablas independientes y no se declaren claves foráneas. Los tipos `jsonb` permiten almacenar estructuras JSON, pero no convierten PostgreSQL en una base de datos NoSQL. Conviene confirmar con el docente que esta interpretación se ajusta al requisito académico.

3. Esquema Supabase planteado

El esquema y las restricciones fueron creados por el usuario en Supabase. Se revisó la propuesta proporcionada para estas tablas:

Tabla	Propósito	Datos y reglas principales
`profiles`	Perfil, email y rol	`uuid`, `text`, `timestamptz`; roles `ADMIN` y `STANDARD_USER`; perfil asociado al identificador de autenticación, sin FK declarada.
`tasks`	Tareas y datos asociados	`uuid`, `text`, `numeric`, `jsonb`, `timestamptz`; estados y prioridades restringidos; horas no negativas; arrays JSON validados; `created_by_user_id` por defecto usa `auth.uid()`.
`activity_logs`	Auditoría de eventos	`uuid`, `text`, `jsonb`, `timestamptz`; acción, usuario, tarea y detalles del evento.

La propuesta incluye índices para filtros habituales, actualización automática de `updated_at`, políticas RLS por rol/propiedad y la creación automática de perfiles al registrarse. Estas políticas se basan en el SQL compartido por el usuario; aún falta probarlas contra el proyecto remoto con cuentas de distintos roles.

4. Auditoría de actividad y seguridad

Se detectó que la política cliente `activity_insert` permitía que un usuario autenticado insertara eventos de auditoría directamente. Eso permitiría fabricar logs desde el cliente y no es necesario si la base de datos genera los eventos automáticamente.

La migración `supabase/migrations/20261027175000_restrict_activity_log_inserts.sql`:

1. Elimina `activity_insert`.
2. Revoca `INSERT` directo en `activity_logs` a `PUBLIC`, `anon` y `authenticated`.
3. Crea o reemplaza `public.log_task_activity()` como trigger `SECURITY DEFINER`, con `search_path` vacío.
4. Registra creación, modificación y eliminación de tareas en `activity_logs`.
5. En actualizaciones, identifica los nombres de los campos que cambiaron y omite `updated_at`.
6. Vincula la función a un trigger posterior a `INSERT`, `UPDATE` y `DELETE` en `tasks`.

Estado: el usuario confirmó que ejecutó el script en Supabase. El proyecto ahora consulta `activity_logs` para mostrar el historial y refresca tareas e historial tras eventos Realtime en `tasks`. La verificación funcional de creación, actualización y eliminación en la base remota sigue pendiente; además, la interfaz aún no tiene todos esos flujos CRUD para probarlos desde la app.

5. Interfaz implementada

La página de inicio contiene un panel responsive con navegación, búsqueda, tarjetas resumen, lista de tareas, historial de auditoría y dos visualizaciones:

- Gráfico SVG de líneas para la actividad semanal.
- Gráfico SVG de dona para la distribución por estado.

La interfaz permite buscar por título, descripción y etiquetas; filtrar por estado; actualizar el estado de una tarea en Supabase; y refrescar datos mediante Realtime. Las tarjetas resumen de tareas y horas se calculan a partir de las filas visibles para RLS. El esquema compartido no tiene columna de fecha límite, por lo que las tareas se muestran sin fecha límite. La gráfica semanal permanece ilustrativa y el botón para crear tareas sigue deshabilitado.

Se añadieron pantallas de registro e inicio de sesión con correo y contraseña. El registro envía `full_name` en los metadatos de Auth para que el trigger de perfiles pueda consumirlo. La sesión se conserva con Supabase Auth y `/home` queda protegido por un guard; el perfil permite cerrar sesión.

Se mejoraron el idioma y título del documento, el zoom en dispositivos móviles, el contraste de textos, los nombres accesibles de botones y los estilos de foco. Se redujeron estilos globales innecesarios.

6. Preparación de la conexión Supabase

- Se instaló `@supabase/supabase-js`.
- Se creó `SupabaseClientService` para cargar `/assets/supabase-config.json`, validar la configuración y reutilizar el cliente.
- La configuración local se excluye de Git mediante `.gitignore`.
- Existe `src/assets/supabase-config.example.json` como plantilla, sin credenciales reales.
- Se confirmó que el archivo local no está versionado y que Git lo ignora.
- La conexión al endpoint de salud de autenticación respondió **HTTP 200**.
- La consulta REST a `public.tasks` con las columnas esperadas respondió **HTTP 200**.
- Las pruebas usan valores ficticios y no contienen credenciales del proyecto.

La clave configurada es de cliente (anon/publishable); no debe confundirse con una clave `service_role`, que nunca debe exponerse en una aplicación frontend. Aunque una clave pública puede viajar al navegador, el control de acceso real depende de RLS y de las políticas de la base de datos.

7. Validación realizada

Validación	Resultado
Build de producción Angular	Correcto
Pruebas unitarias	18 aprobadas en 4 archivos
Lint	Correcto
Auditoría de dependencias de producción	0 vulnerabilidades reportadas
Lighthouse móvil de producción	Rendimiento 97/100
Lighthouse móvil de producción	Accesibilidad 100/100
Lighthouse móvil de producción	Buenas prácticas 100/100
Revisión responsive	Sin desbordamiento horizontal a 390 px
Prueba funcional en navegador	`/home` redirige a autenticación sin sesión; formulario de registro presenta nombre, email y contraseña
Supabase Auth health	HTTP 200
Supabase REST de tareas	HTTP 200

La medición Lighthouse se realizó sobre la compilación de producción con compresión y caché de archivos estáticos. El resultado del servidor de desarrollo no se usa como referencia de rendimiento.

Las 18 pruebas cubren la creación y validación del cliente, la consulta y conversión de tareas y logs, errores de permisos RLS, la suscripción a Realtime y los comportamientos del panel: filtros, búsqueda, actualización de estado, métricas y gráficos.

8. Estado actual y trabajo pendiente

Área	Estado	Siguiente paso
Tablas y restricciones	Creadas por el usuario en Supabase	Verificar columnas y políticas con pruebas reales.
`activity_insert` / auditoría	Migración aplicada según el usuario; interfaz lee historial	Comprobar eventos reales tras crear, actualizar y borrar tareas.
Cliente Supabase	Conectado; Auth y REST respondieron HTTP 200	Verificar flujos con una cuenta no productiva del proyecto.
Registro e inicio de sesión	Implementados con email/password y perfil automático esperado	Probar registro, confirmación por correo según ajustes Auth y login real.
Perfil y roles	El trigger del perfil fue indicado por el usuario; se usa `full_name` de metadatos para mostrar identidad	Comprobar perfil y roles reales con RLS.
Guard de rutas	Protege `/home` y redirige a `/auth` sin sesión	Probar persistencia y expiración de sesión con cuenta de prueba.
Tareas remotas	Lectura, filtros y actualización de estado; horas reales; error visible al fallar RLS	Implementar alta y eliminación, y validar ownership con usuario estándar/admin.

Área	Estado	Siguiente paso
Realtime	Suscripción a eventos de `tasks`; el panel refresca tareas e historial	Probar con dos sesiones autenticadas y revisar la configuración de publicación/RLS.
Pruebas de autorización	Pendientes	Probar usuario anónimo, usuario estándar y administrador, incluyendo intentos denegados.

9. Orden recomendado para continuar

1. Probar login y registro con una cuenta de prueba y confirmar la creación de su perfil automático.
2. Verificar el trigger de auditoría creando y actualizando una tarea; confirmar que el usuario no puede insertar logs directamente.
3. Implementar el formulario y política RLS necesaria para crear tareas, si esa operación corresponde al alcance acordado.
4. Probar permisos de lectura y actualización para usuario asignado, creador, otro usuario estándar y administrador.
5. Validar Realtime con dos sesiones autenticadas y confirmar los cambios visibles en el otro navegador.
6. Implementar eliminación si forma parte del alcance, probando el evento de auditoría correspondiente.
7. Conectar los datos reales del gráfico semanal o mantenerlo claramente marcado como ilustrativo.
8. Realizar una prueba final de sesión, RLS, CRUD implementado y manejo de errores en el navegador.

10. Referencias del proyecto

- `src/app/home/home.page.ts` y `home.page.html`: panel y comportamiento visual.
- `src/app/home/home.page.spec.ts`: pruebas del panel.
- `src/app/core/models/task.model.ts`: tipos de tarea de la interfaz.
- `src/app/core/supabase/supabase-client.service.ts`: inicialización del cliente Supabase.
- `src/app/core/auth/auth.service.ts` y `auth.guard.ts`: sesión y protección de rutas.
- `src/app/auth/`: formularios de registro e inicio de sesión.
- `src/app/core/tasks/task-data.service.ts`: consultas, actualización de tareas, historial y canal Realtime.
- `src/app/core/tasks/task-data.service.spec.ts`: pruebas del repositorio de tareas e historial.
- `src/app/core/models/task-row.model.ts`: filas de tareas/logs recibidas de Supabase.
- `src/app/core/supabase/supabase-client.service.spec.ts`: pruebas del cliente con configuración ficticia.
- `src/assets/supabase-config.example.json`: plantilla de configuración.
- `src/assets/supabase-config.json`: configuración local excluida de Git; no incluir sus valores en documentación o capturas.
- `supabase/migrations/20261027175000_restrict_activity_log_inserts.sql`: migración de auditoría ejecutada según el usuario.

Conclusión: la aplicación ya tiene autenticación, guard de ruta, lectura de tareas e historial, actualización de estado y Realtime conectados a Supabase; build, lint y 18 pruebas pasan. La migración de auditoría se ejecutó según el usuario, aunque todavía falta verificar los eventos reales en la base. El trabajo siguiente es probar Auth con una cuenta de prueba, verificar RLS y completar la creación/eliminación de tareas si entran en el alcance.